



Core Java

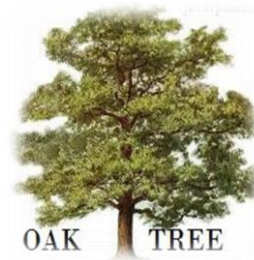
Introduction to Java

Outline

- Introduction to Java 21
 - History of Java, Salient features of Java, Java Development Kit, JDK's Command-line tools, Application Programming Interface, Bytecode, JShell.
 - A Simple Java Program, Data types, Variables and Constants, Operators, Type conversion and casting, Automatic type promotion, Strings & String Buffer, Input & Output, Control Statements, Arrays, Generic Array List
 - Classes, Objects, Use a predefined class, Define own Class, Object Construction, Access Specifiers, Method Parameters, Static Fields & Methods, Constructors, Destructors, Packages, Documentation Comments.

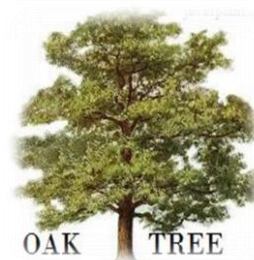
History of Java

- **James Gosling, Mike Sheridan,** and **Patrick Naughton** initiated the **J**ava language project in June 1991. The small team of **Sun engineers** called **Green Team**.
- Firstly, it was called "**Greentalk**" by James Gosling, and file extension was **.gt**.
- **J**ava was originally **designed for small, embedded systems in electronic appliances** like set-top boxes, but it was too advanced technology for the digital cable television industry at the time.



History of Java

- After that, it was called **Oak** and was developed as a part of the Green project. **J**ava team members initiated this project to develop a language for digital devices.
- Later, **J**ava technology was incorporated by **Netscape** as it was suited for networking.

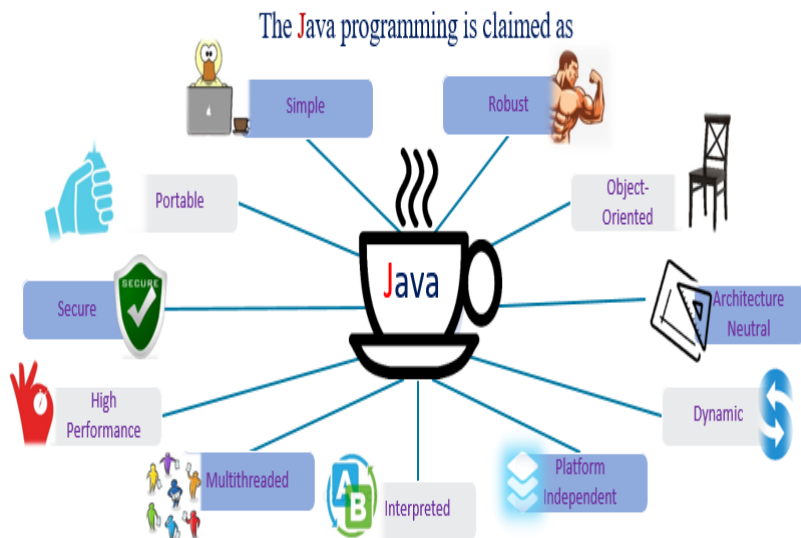


Why Java is named Java?

- **Java** was called **Oak** as it is a symbol of strength and chosen as a national tree of many countries like U.S.A., France, Germany, Romania, etc.
- The team wanted something that reflected the essence of the technology: revolutionary, dynamic, lively, cool, unique, and easy to spell and fun to say.
- In 1995, Oak was renamed as **Java**
- **Java is an island of Indonesia where first coffee** was produced (called java coffee).
- In 1995, Time magazine called **Java one of the Ten Best Products of 1995**.
- **JDK** (Java Development Kit) 1.0 released in January 23, 1996.



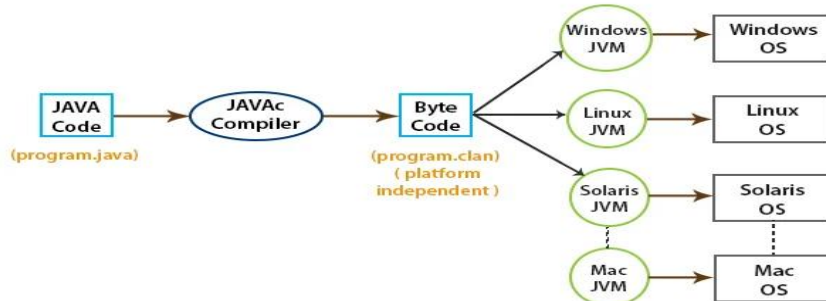
Java



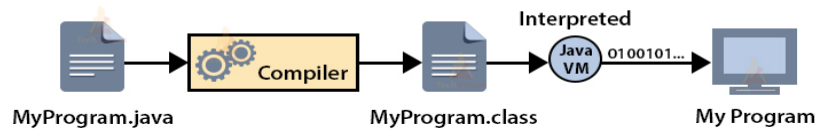
Features of Java



Platform Independence in Java

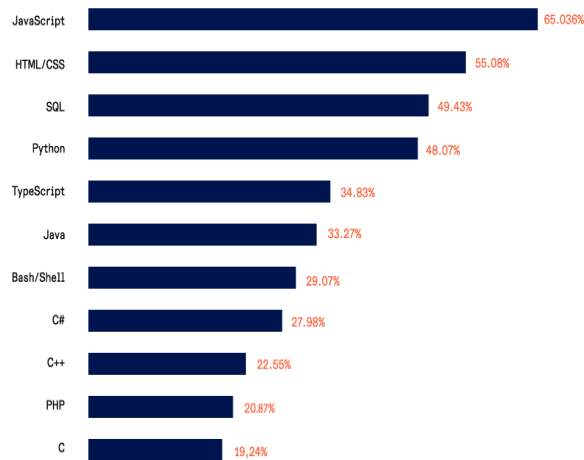


Working of Java Virtual Machine



Current popularity

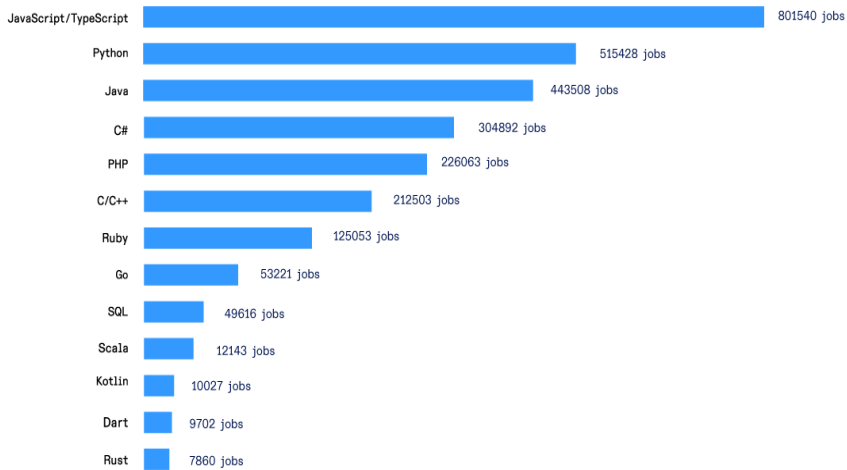
Most used programming languages among developers



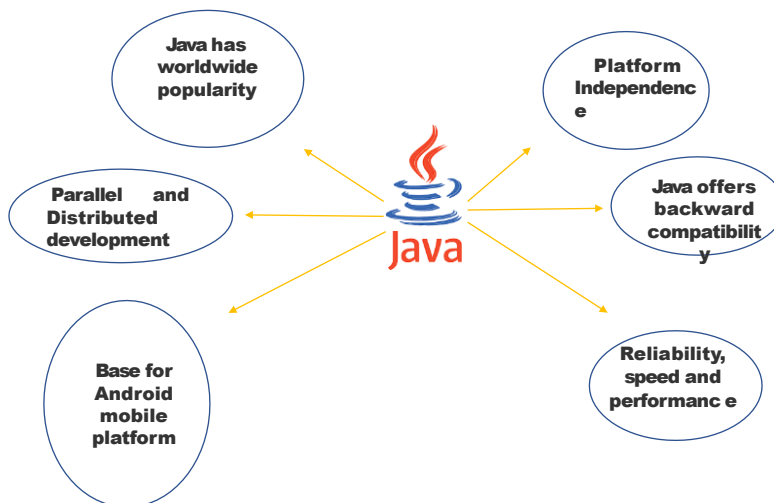
Source: Statista

Current popularity

Most Demanded Programming Languages by Number of Jobs

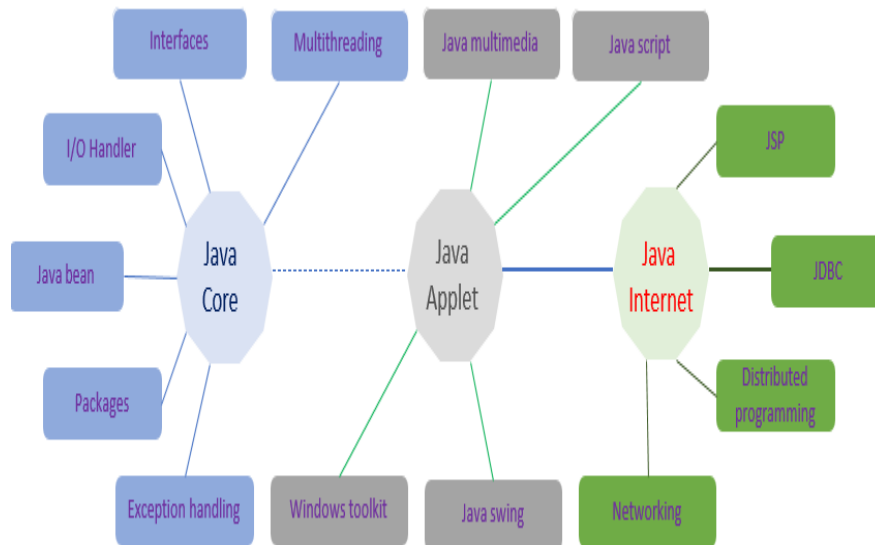


Current popularity



Java has consistently been more popular than any other programming language.

Java Programming



Components of Java

- **JVM (Java Virtual Machine)**
 - It is an abstract machine. It is a specification that provides a run-time environment in which the bytecode can be executed. It follows three notations:
 - **Specification:** It is a document that describes the implementation of the JVM. It is provided by Sun and other companies.
 - **Implementation:** It is a program that meets the requirements of JVM specification.
 - **Runtime Instance:** An instance of JVM is created whenever you write a command on the command prompt and run the class.

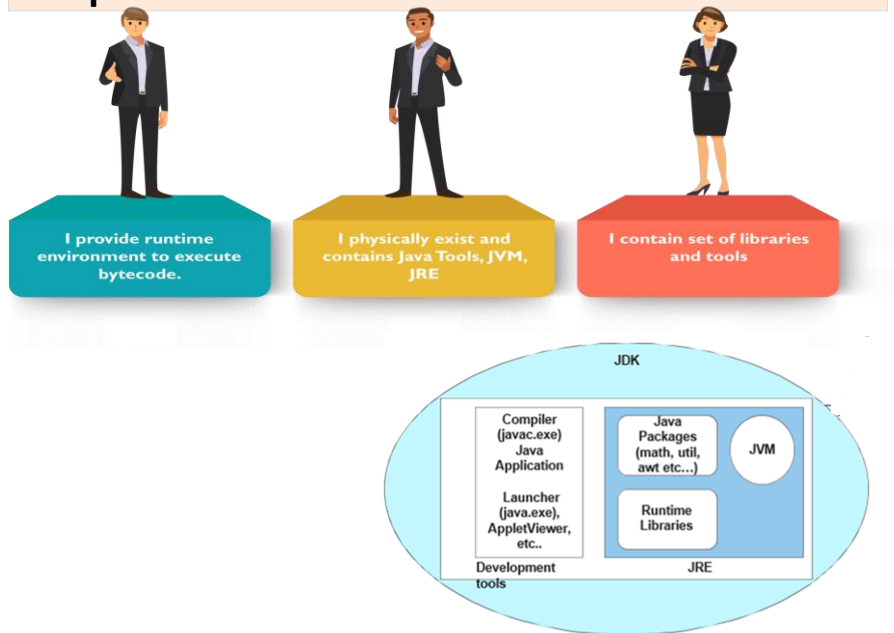
Components of Java

- **JRE (Java Runtime Environment)**
 - JRE refers to a runtime environment in which bytecode can be executed. It implements the JVM and provides all the class libraries and other support files that JVM uses at runtime. So JRE is a software package that contains what is required to run a program. Basically, it's an implementation of the JVM which physically exists.

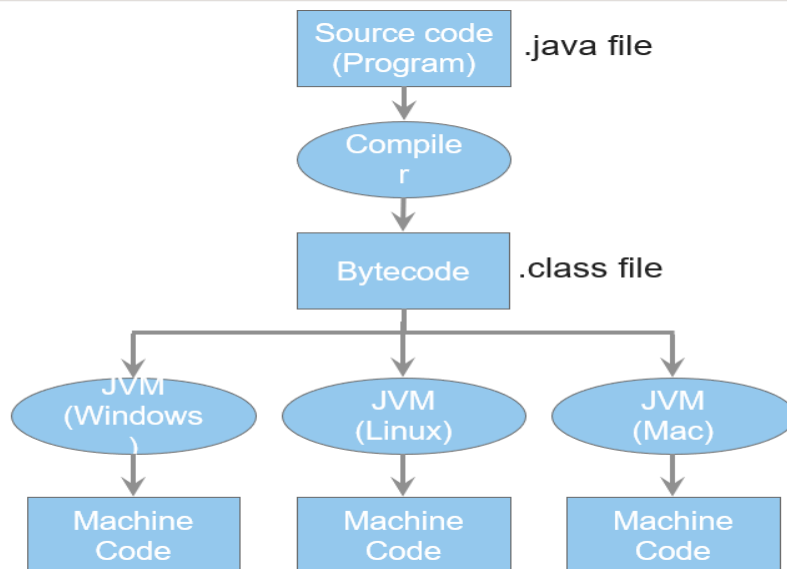
Components of Java

- **JDK(Java Development Kit)**
 - It is the tool necessary to:-
 - Compile
 - Document
 - Package Java programs.
 - The JDK completely includes JRE which contains tools for programmers. The Development Kit is provided free of charge. Along with JRE, it includes an interpreter/loader, a compiler (javac), an archiver (jar), a documentation generator and other tools needed in Java development. In short, it contains JRE + development tools.

Components of Java



How java becomes platform independent



Editions of Java Platform

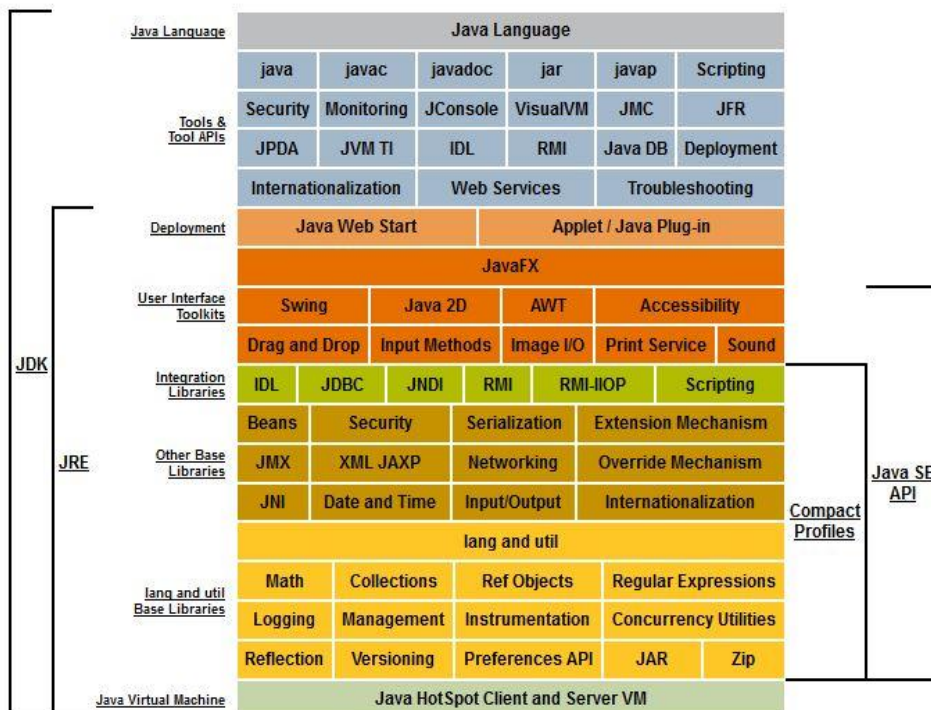
- **Java SE (Standard Edition):** This edition is used to develop client-side applications. It is used to develop applications for desktop, communication, and user interface.
- **Java EE (Enterprise Edition):** This edition is used to develop server-side applications such as Java servlets, JavaServer Pages (JSP), and JavaServer Faces (JSF).
 - In other words, it is used to develop web-based, messaging, distributed, and enterprise applications.
- **Java ME (Micro Edition):** This edition is used to develop applications for mobile devices, such as cell phones. It is also used to develop Personnel Digital Assistants, Setup Box, and printers applications.

Java

- The list of major Java SE (Standard Edition) and JDK versions with their General Availability (GA) dates is as follows:
- | Version | GA Date | Codename | Long-Term Support (LTS)? |
|-----------------|----------------|-------------------|--------------------------|
| JDK 1.0 | January 1996 | Oak | No |
| JDK 1.1 | February 1997 | — | No |
| J2SE 1.2 | December 1998 | Playground | No |
| J2SE 1.3 | May 2000 | Kestrel | No |
| J2SE 1.4 | February 2002 | Merlin | No |
| J2SE 5.0 (1.5) | September 2004 | Tiger | No |
| Java SE 6 (1.6) | December 2006 | Mustang | No |
| Java SE 7 (1.7) | July 2011 | Dolphin | No |
| Java SE 8 (1.8) | March 2014 | Spider (informal) | Yes |
| Java SE 9 | September 2017 | — | No |
| Java SE 10 | March 2018 | — | No |
| Java SE 11 | September 2018 | — | Yes |

Java

Version	GA Date	Codename	Long-Term Support (LTS)?
Java SE 12	March 2019	—	No
Java SE 13	September 2019	—	No
Java SE 14	March 2020	—	No
Java SE 15	September 2020	—	No
Java SE 16	March 2021	—	No
Java SE 17	September 2021	—	Yes
Java SE 18	March 2022	—	No
Java SE 19	September 2022	—	No
Java SE 20	March 2023	—	No
Java SE 21	September 2023	—	Yes
Java SE 22	March 2024	—	No
Java SE 23	September 2024	—	No
Java SE 24	March 2025	—	No
Java SE 25	September 2025	—	Yes



Java IDE(Integrated Development Environment)

Top Java IDE Tools That Every Developer Must Know!



Tools available for Java programming

- Java Software Developer's Kit (SDK) : Contains the basic tools and libraries necessary for creating, testing, documenting and executing Java programs.
- There are seven main programs in SDK
 - *javac* – the Java Compiler
 - *java* – the Java Interpreter
 - *javadoc* – generates documentation in HTML
 - *appletviewer* – the Java Interpreter to execute Java applets
 - *jdb* – the Java Debugger to find and fix bugs in Java programs
 - *javap* – the Java Disassembler to displays the accessible functions and data in a compiled class; it also displays the meaning of byte codes
 - *javah* – to create interface between Java and C routines

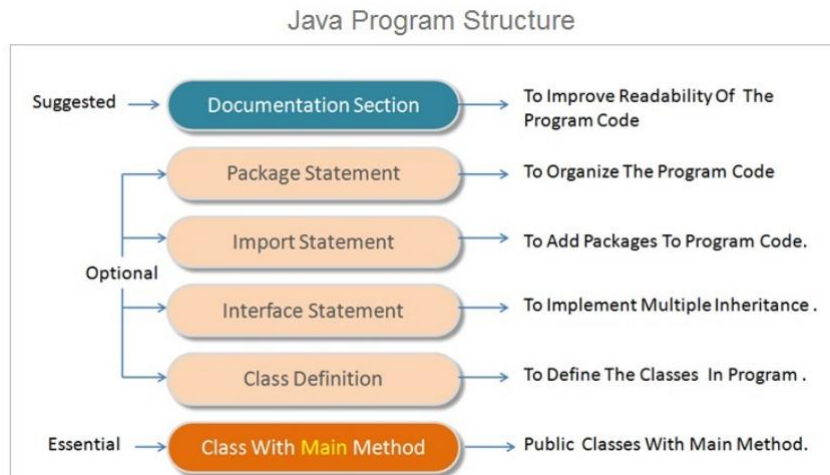
Packages in Java

- **API (Application Programming Interface) in Java SDK**
 - The API enables Java programmers to develop varieties of applets and applications
 - It contains **nine** packages
 - *java.applet* – for applet programming
 - *java.awt* – the **Abstract Windowing Toolkit** for designing GUI like *Button*, *Checkbox*, *Choice*, *Menu*, *Pannel*, etc.
 - *java.io* – file input/output handling
 - *java.lang* – provides useful classes like to handle *Object*, *Thread*, *Exception*, *String*, *System*, *Math*, *Float*, *Integer*, etc.

Packages in Java

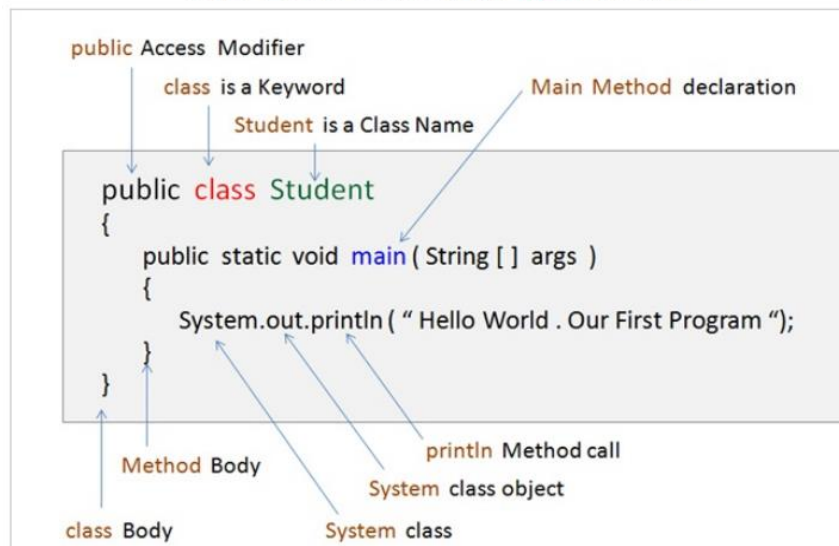
- *java.net* – classes for **network programming**; supports TCP/IP networking protocols
- *java.util* – it contains miscellaneous classes like *Vector*, *Stack*, *List*, *Date*, *Dictionary*, *Hash* etc.
- *javax.swing* – for designing graphical user interface (GUI)
- *java.sql* – for database connectivity (**JDBC**)

Java Program Structure



Java Program

Java Public Class With Main Method



Compiling and running Java Program

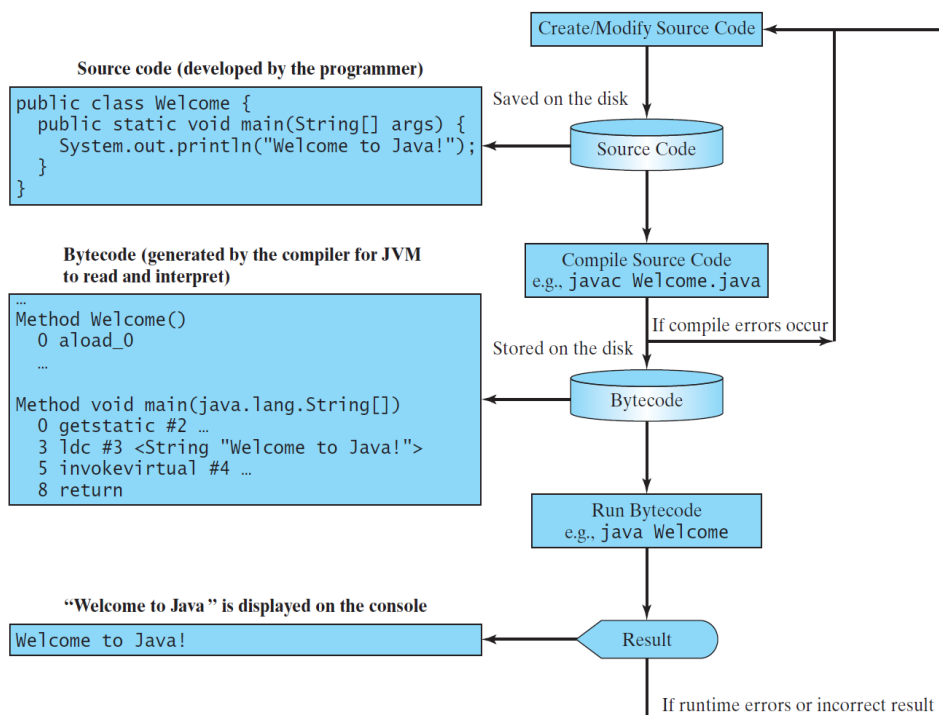
```
C:\>cd java programs
C:\Java Programs>javac HelloWorld.java
C:\Java Programs>java HelloWorld
Hello World
C:\Java Programs>_
```

First compile your Java program using java compiler (javac.exe) .

Execute the **dot class** file .

Compile - **javac** HelloWorld.java

Execute - **java** HelloWorld

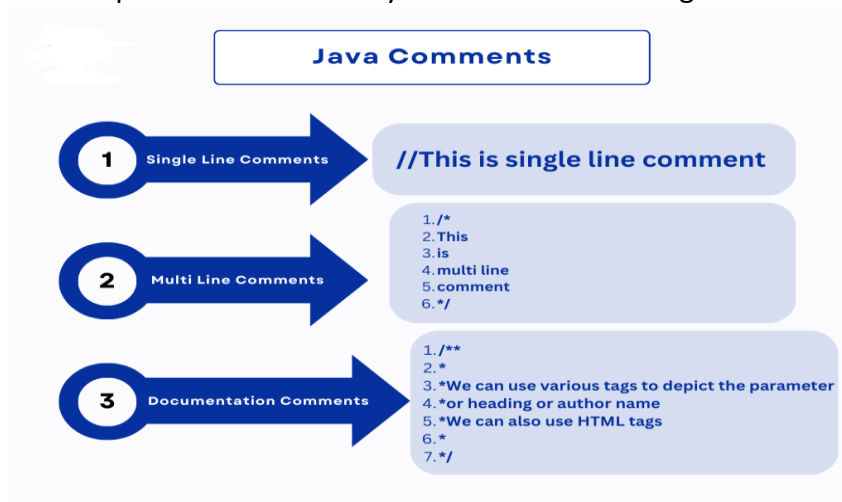


Comment in java

- Comments are lines of text added to the code to explain what the code does.
- These comments are ignored by the compiler, meaning they don't affect how the program runs. They are solely for the benefit of developers to understand the code better.
- Comments play a crucial role in programming for several reasons:
 - **Code Explanation:** Comments help explain the purpose of variables, methods, or complex algorithms, making it easier for developers to understand the code, especially when revisiting it after some time.
 - **Documentation:** Comments can serve as documentation for other developers who may work on the codebase in the future. Good documentation helps maintain and update code effectively.

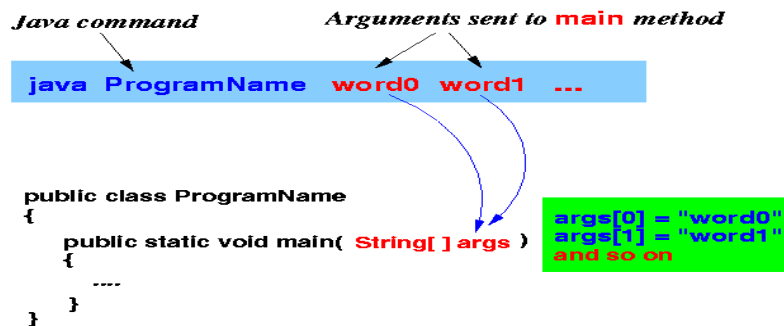
Comment in java

- **Debugging:** Comments can be used to temporarily disable lines of code during debugging without deleting them. This practice can be handy when troubleshooting issues.



Command line arguments

- The command line arguments in Java allow the programmers to pass arguments (i.e. values) to the main() method during the execution of the program from the outside.
- It represents the arguments passed to the main() method. To catch and store these arguments, the main() method has a parameter, String args[] as:



Command line arguments

- In the main() method, args[] is a one-dimensional array of string type.
- When we pass command-line arguments, they are treated as strings and passed to the main function in the string array argument.
- The arguments have to be passed as space-separated values.
- We can pass strings and primitive data types as command-line arguments. The arguments will be converted to strings and passed into the main method string array argument.
- It is used because it allows us to provide input at runtime without modifying the whole program.
- It helps to run programs automatically by giving them the needed information from outside.

Working of Command line arguments

- Command-line arguments in Java are space-separated values passed to the `main(String[] args)` method.
- JVM wraps them into the `args[]` array, where each value is stored as a string (e.g., `args[0]`, `args[1]`, etc.).
- The number of arguments can be checked using `args.length`.
- **Note** : For any arithmetic operation on command line arguments convert string to appropriate data type and use.

Command line arguments Example

```
public class CommandLineArguments {
    public static void main(String[] args) {
        System.out.println("Number of Command Line
        Argument = "+args.length);
        for(int i = 0; i< args.length; i++) {
            System.out.println(String.format("Command
            Line Argument %d is %s", i, args[i]));
        } // end of for loop
    } // end of main
} // end of class
```

- If we run this class without any arguments, the output will be as follows.
- PS> java CommandLineArguments.java
Output Number of Command Line Argument = 0

Command line arguments Example

- If we run this class without any arguments, the output will be as follows.
- PS> java CommandLineArguments~~java~~
 Output Number of Command Line Argument = 0
- Now, let's pass some arguments to the main class. We have to pass the arguments as space-separated values.
- PS> java CommandLineArguments~~java~~ "A" "B" "C"
- Output
 Number of Command Line Argument = 3
 Command Line Argument 0 is A
 Command Line Argument 1 is B
 Command Line Argument 2 is C

Command line arguments Example

- PS> java CommandLineArguments~~java~~ 1 2 3
- Output
 Number of Command Line Argument = 3
 Command Line Argument 0 is 1
 Command Line Argument 1 is 2
 Command Line Argument 2 is 3

Command line arguments Example

- **Reading Numeric and Finding the Sum of Two Numbers**

```
public class Example2 {
    public static void main(String[] args) {
        if (args.length >= 2) {
            int sum = Integer.parseInt(args[0]) + Integer.parseInt(args[1]);
            System.out.println("Sum: " + sum);
        } else {
            System.out.println("Two numeric arguments are required.");
        }
    }
}
```

Execution: java Example 10 20

Output : Sum: 30

Command line arguments Example

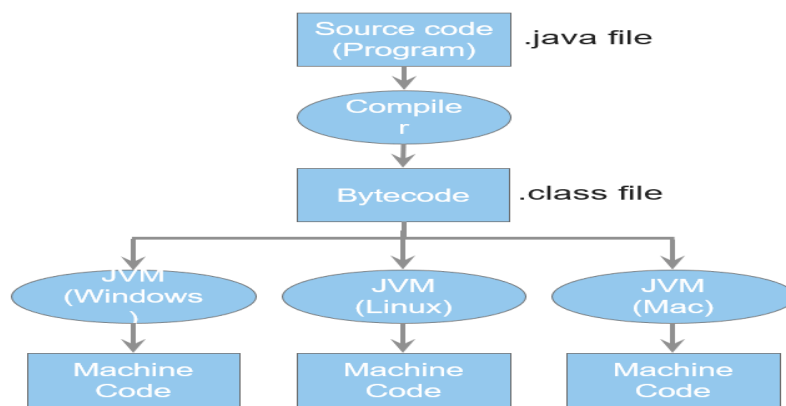
- The method `parseInt()` belongs to the `Integer` class under `java.lang` package. This method is used to convert a string value to an integer.
- `Integer.parseInt(args[0])` and `Integer.parseInt(args[1])`:
- Converts the command-line arguments (strings) to integers.
- `args[0]` and `args[1]` are the first and second elements of the `args` array.
- **Note:** Similarly, we can use the `parseDouble()` and `parseFloat()` method to convert the string into double and float respectively.
- **Note:** If the arguments cannot be converted into the specified numeric value then an exception named `NumberFormatException` occurs.

Bytecode in Java

- **Bytecode in Java** is a highly optimized set of instructions for the *Java Virtual Machine* (JVM) that reads and interprets to run the java program.
- A bytecode is a binary program code that can only run on JVM. In other words, it is a machine language (code) for JVM in the form of *.class* file, but it is not machine specific because it is not a native code.
- Byte code acts as an intermediate language that is platform (machine) independent. It is generated by Java interpreter that can be directly run by a real machine.
- **Characteristics of Bytecode in Java:**
 - Byte code is independent of processor, i.e., Java program can be executed on any processor architecture.
 - It does not depend on operating systems such as Windows, Linux, and Mac OS.

Bytecode in Java

- When we write a java program, the source code (in the form of *.java* file) is compiled by Java compiler and converted into byte code in the form of a *.class* file.



Difference between Bytecode vs Machine code

- The main difference between the byte code and machine code is that byte code can be run only on JVM whereas machine code is a set of instructions in machine language that can be directly run by the CPU.

Difference between .java and .class

- Java source code file (with a .java extension) are compiled into bytecode (with a .class extension), which is then interpreted and executed by a Java processor called JVM.

JShell

- JShell, or Java Shell, is an interactive Read-Evaluate-Print Loop (REPL) tool introduced in Java 9.
- It provides a command-line interface for learning and experimenting with Java code snippets without the need for a full project setup, compilation, and execution cycle.
- **How to Use JShell:**
 - Open a terminal or command prompt.
 - Type jshell and press Enter: to start the JShell session.
 - Enter Java code snippets or JShell commands: at the jshell> prompt.
- jshell> System.out.println("Hello, JShell!");
- Hello, JShell!

JShell

- `jshell> int x = 10;`
`x ==> 10`
- `jshell> x * 2;`
`$3 ==> 20`
- `jshell> /list`
- Type `/exit` to close the JShell session.
- Built-in Commands: JShell includes various commands (prefixed with `/`) for managing the session, listing variables and methods, importing packages, and more (e.g., `/help`, `/list`, `/imports`).

JShell (adding two numbers)

```
PS D:\java programs testing> Jshell
| Welcome to JShell -- Version 25.0.1
| For an introduction type: /help intro

jshell> int a = 10;
a ==> 10

jshell> int b = 20;
b ==> 20

jshell> int sum = a + b;
sum ==> 30

jshell> System.out.println("Sum = " + sum);
Sum = 30

jshell> /exit
| Goodbye
PS D:\java programs testing> █
```

JShell (calling functions)

```
jshell> int divide(int x, int y){
    ...>     return x/y;
    ...> }
|   created method divide(int,int)

jshell> divide(10,2)
$2 ==> 5

jshell> /list

    1 : int divide(int x, int y){
        return x/y;
    }
    2 : divide(10,2)

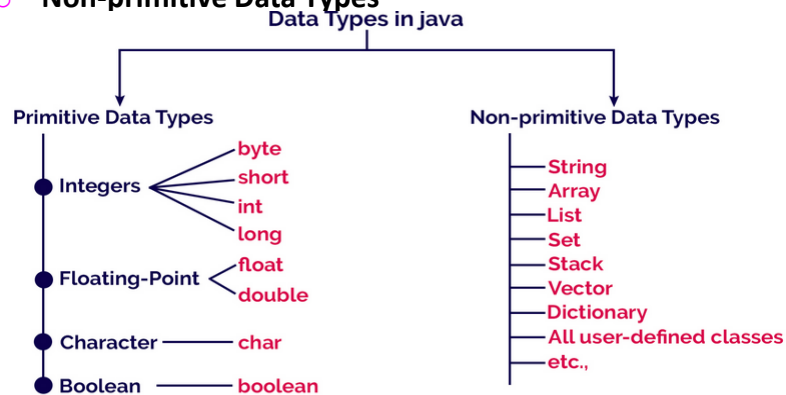
jshell> █
```

JShell

- jshell> int x = 10;
x ==> 10
- jshell> x * 2;
\$3 ==> 20
- jshell> /list
- Type /exit to close the JShell session.
- Built-in Commands: JShell includes various commands (prefixed with /) for managing the session, listing variables and methods, importing packages, and more (e.g., /help, /list, /imports).

Data Types in Java

- The data type is a category of data stored in variables.
- In java, data types are classified into two types and they are as follows.
 - **Primitive Data Types**
 - **Non-primitive Data Types**



Data Types in Java

- **Primitive Data Types** : The primitive data types are built-in data types and they specify the type of value stored in a variable and the memory size.
- The primitive data types do not have any additional methods.
- In java, primitive data types includes **byte**, **short**, **int**, **long**, **float**, **double**, **char**, and **boolean**.

Primitive Data Types

Data type	Meaning	Memory size	Range	Default Value
byte	Whole numbers	1 byte	-128 to +127	0
short	Whole numbers	2 bytes	-32768 to +32767	0
int	Whole numbers	4 bytes	-2,147,483,648 to +2,147,483,647	0
long	Whole numbers	8 bytes	-9,223,372,036,854,775,808 +9,223,372,036,854,775,807	0L
float	Fractional numbers	4 bytes	-	0.0f
double	Fractional numbers	8 bytes	-	0.0d
char	Single character	2 bytes	0 to 65535	\u0000
boolean	unsigned char	1 bit	0 or 1	0 (false)

Primitive Data Type example

```
public class PrimitiveDataTypes {
    byte i;
    short j;
    int k;
    long l;
    float m;
    double n;
    char ch;
    boolean p;
```

Primitive Data Type example

```

public static void main(String[] args) {
    PrimitiveDataTypes obj = new PrimitiveDataTypes();
    System.out.println("i = " + obj.i + ", j = " + obj.j + ", k
    = " + obj.k + ", l = " + obj.l);
    System.out.println("m = " + obj.m + ", n = " + obj.n);
    System.out.println("ch = " + obj.ch);
    System.out.println("p = " + obj.p);
}
}

```

```

i = 0, j = 0, k = 0, l = 0
m = 0.0, n = 0.0
ch =
p = false

```

Data Types in Java

- **Non-primitive Data Types** : In java, non-primitive data types are the reference data types or user-created data types.
- All non-primitive data types are implemented using object concepts. Every variable of the non-primitive data type is an object.
- The non-primitive data types may use additional methods to perform certain operations. The default value of non-primitive data type variable is null.
- In java, examples of non-primitive data types are [String](#), [Array](#), [List](#), [Queue](#), [Stack](#), [Class](#), [Interface](#), etc.

Data Types in Java

- **Non-primitive Data Types** : In java, non-primitive data types are the reference data types or user-created data types.
- All non-primitive data types are implemented using object concepts. Every variable of the non-primitive data type is an object.
- The non-primitive data types may use additional methods to perform certain operations. The default value of non-primitive data type variable is null.
- In java, examples of non-primitive data types are [String](#), [Array](#), [List](#), [Queue](#), [Stack](#), [Class](#), [Interface](#), etc.

Non-primitive Data type example

```
public class NonPrimitiveDataTypes {
    String str;
    public static void main(String[] args) {
        String name = "MCA Class!";
        String wish = "Hello, ";
        NonPrimitiveDataTypes obj = new
                                NonPrimitiveDataTypes();
        System.out.println("str = " + obj.str);
        //using addition method
        System.out.println(wish.concat(name));
    }
}
```

str = null
Hello, MCA Class!

Primitive vs Non-primitive Data type

Primitive Data Type	Non-primitive Data Type
These are built-in data types	These are created by the users
Does not support additional methods	Support additional methods
Always has a value	It can be null
Starts with lower-case letter	Starts with upper-case letter
Size depends on the data type	Same size for all

Keywords

Category	Java Keywords
Access Modifiers	private, protected, public
Class, Method, Variable Modifiers	abstract, class, extends, final, implements, interface, native, new, static, strictfp, synchronized, transient, volatile
Flow Control	break, case, continue, default, do, else, for, if, instanceof, return, switch, while
Package Control	import, package
Primitive Types	boolean, byte, char, double, float, int, long, short
Error Handling	assert, catch, finally, throw, throws, try
Enumeration	enum
Others	super, this, void
Unused	const, goto

Variables and Constant

- A variable is a named memory location used to store a data value. A variable can be defined as a container that holds a data value.
- In java, we use the following syntax to create variables.
- **Syntax :**
 - `data_type variable_name;`
 - `data_type variable_name_1, variable_name_2,...;`
 - `data_type variable_name = value;`
 - `data_type variable_name_1 = value, variable_name_2 = value,...;`

Variables and Constant

- In java programming language variables are classified as follows.
 - **Local variables**
 - **Instance variables or Member variables or Global variables**
 - **Static variables or Class variables**
 - **Final variables**
- **Local variables :**
 - The variables declared inside a method or a block are known as local variables.
 - A local variable is visible within the method in which it is declared.
 - The local variable is created when execution control enters into the method or block and destroyed after the method or block execution completed.

Variables and Constant

- **Instance variables or Member variables or Global variables :**
 - The variables declared inside a class and outside any method, constructor or block are known as instance variables or member variables.
 - These variables are visible to all the methods of the class.
 - The changes made to these variables by method affects all the methods in the class.
 - These variables are created separate copy for every object of that class.

Variables and Constant

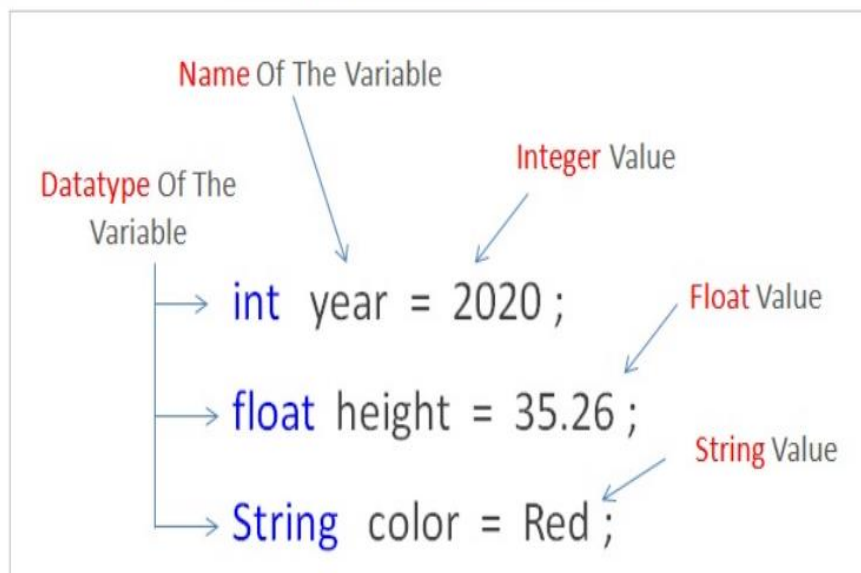
- **Static variables or Class variables :**
 - A **static** variable is a variable that declared using static keyword.
 - The instance variables can be static variables but local variables can not.
 - Static variables are initialized only once, at the start of the program execution.
 - The static variable only has one copy per class irrespective of how many objects we create.
 - The static variable is access by using class name.

Variables and Constant

- **Final variables :**
 - A final variable is a variable that declared using **final** keyword.
 - The final variable is initialized only once, and does not allow any method to change it's value again.
 - The variable created using final keyword acts as constant.
 - All variables like local, instance, and static variables can be final variables.

Variables and Constant

Java Variable Declaration

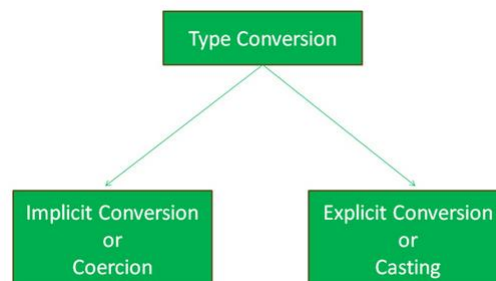


Java Operators

	Java Operator	Examples
1	Arithmetic	<code>+, -, /, *, %</code>
2	Unary	<code>++, --, !</code>
3	Assignment	<code>=, +=, -=, *=, /=, %=, ^=</code>
4	Relational	<code>==, !=, <, >, <=, >=</code>
5	Logical	<code>&&, </code>
6	Ternary	<code>(Condition) ? (Statement1) : (Statement2) ;</code>
7	Bitwise	<code>&, , ^, ~</code>
8	Shift	<code><<, >>, >>></code>

Type conversion and casting

- Converting a value from one type to another type (data type) is known as type conversion.
- Type conversion is of two types based on how the conversion is performed:
 - Implicit conversion (also known as automatic conversion or coercion)
 - Explicit conversion (also known as type casting)



Implicit conversion

- This type of conversion is performed automatically by Java due to performance reasons. Implicit conversion is not performed at all times.
- There are two rules to be satisfied for implicit conversion to take place. They are:
 - The source and destination types must be compatible with each other.
 - The size of the destination type must be larger than the source type.
- For example, Java will automatically convert a value of byte into int type in expressions since they are both compatible and int is larger than byte type.
- Since a smaller range type is converted into a larger range type this conversion is also known as widening conversion. Characters can never be converted to boolean type. Both are incompatible.

Implicit conversion example

- `int x = 20;`
- `long y = x; // Automatic conversion`
- `byte z = x; // Type mismatch: cannot convert from int to byte.`

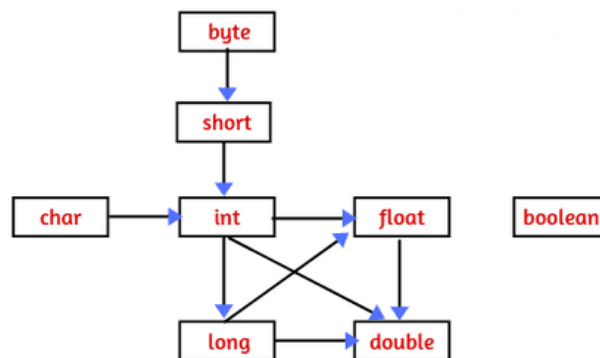


Fig: Automatic type conversion that Java allows.

Automatic Type Promotion Rules in Expression

- If byte, short, and int are used in a mathematical expression, Java always converts the result into an int.
- If a single long is used in the expression, the whole expression is converted to long.
- If a float operand is used in an expression, the whole expression is converted to float.
- If any operand is double, the result is promoted to double.
- Boolean values cannot be converted to another type.
- Conversion from float to int causes truncation of the fractional part which represents the loss of precision. Java does not allow this.
- Conversion from double to float causes rounding of digits that may cause some of the value's precision to be lost.

Automatic Type Promotion Rules in Expression

- Conversion from long to int is also not possible. It causes the dropping of the excess higher order bits.
- For example, when you will perform the addition of an int and a double, it gives the result in a double, and the subtraction of long from float results in float.

Explicit conversion

- There may be situations where you want to convert a value having a type of size less than the destination type size.
- In such cases Java will not help you. You have to do it on your own explicitly. That is why this type of conversion is known as explicit conversion or casting as the programmer does this manually.
- Syntax for type casting is as shown below:
(destination-type) value
- An example for type casting is shown below:

```
int a = 10;
byte b = (byte) a;
```

 - In the above example, we are forcing an integer value to be converted into a byte type.

Explicit conversion

- For type casting to be carried out both the source and destination types must be compatible with each other. For example, you can't convert an integer to boolean even if you force it.
- In the above example, size of source type *int* is 32 bits and size of destination type *byte* is 8 bits.
- Since we are converting a source type having larger size into a destination type having less size, such conversion is known as **narrowing conversion**.
- A type cast can have unexpected behavior. For example, if a *double* is converted into an *int*, the fraction component will be lost.

Type promotion in expressions

- In addition to assignment statements, there is another place where type conversion can occur. It is in expressions.
- An expression is a collection of variables, values, operators and method calls which evaluate to a single value.
- Type promotion rules of Java for expressions are listed below:
 - All char, short and byte values are automatically promoted to int type.
 - If at least one operand in an expression is a long type, then the entire expression will be promoted to long.
 - If at least one operand in an expression is a float type, then the entire expression will be promoted to float.
 - If at least one operand in an expression is a double type, then the entire expression will be promoted to double.

Example

```
class Sample{
    public static void main(String[] args)
    {
        int i = 1000000;
        char c = 'z';
        short s = 200;
        byte b = 120;
        float f = 3.45f;
        double d = 1.6789;
        double result = (f * b) + (i / c)
        - (d * s);
        System.out.println("Result =
        "+result);
    }
}
```

Output of the above program is: Result = 8274.22

Disadvantage of Explicit Type Casting in Java

- When you will use type casting in Java programming, you can lose some information or data.
- Accuracy can be lost while using type casting.
- When a double is cast to an int, the fractional part of a double is discarded, which causes the loss of fractional part of data.

Difference between Implicit Casting and Explicit Casting in Java

- Implicit type casting is done internally by Java compiler whereas, explicit type casting is done by the programmer. Java compiler does not perform it automatically.
- In explicit casting, cast operator is needed whereas, no need for any operator needs in the case of implicit type casting.
- If we perform explicit type casting in a program, we can lose information or data, but in the case of implicit type casting, there is no loss of data.
- Accuracy is not maintained in explicit type casting whereas, there is no issue of accuracy in implicit type conversion.
- Implicit type conversion is safe, but explicit type casting is not safe.

Strings & String Buffer

- A string is a sequence of characters. In C/C++ programming languages, it represents an array of characters, where the last character will be the null character (\0) that represents the end of the string.
- But in Java, String is an object of String class that represents a string of characters. For example, "Pencil" is a string of 6 characters.
- String class is used to create a string object.
- It is a predefined immutable class that is present in **java.lang package**. But in Java, all classes are also considered as a data type. So, you can also consider a string as a data type.

How to create Strings object

- To handle string data in Java, we need an object of string class. Basically, there are three ways to create a string object in Java.
 - By string literal. Example : `String s = "Hello";`
 - By new keyword. Example : `String s = new String("Hello");`
 - By converting character arrays into strings
 - Example : `char arr[ ] = {'j','a','v','a'};`
`String s = new String(arr);`

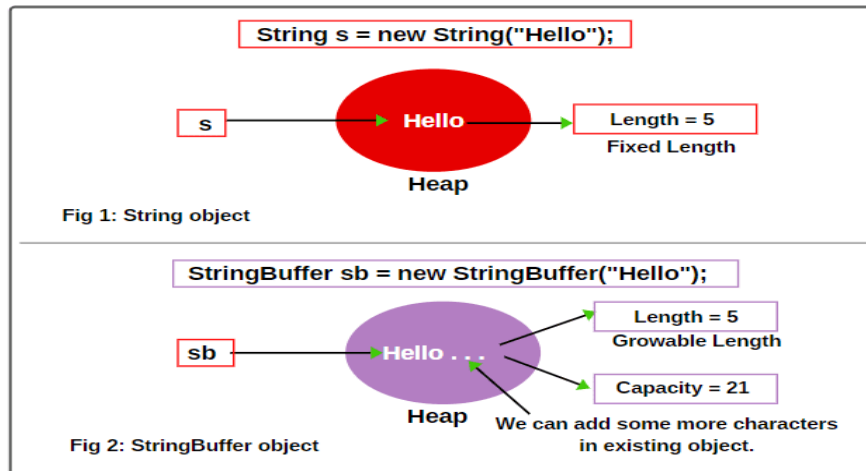
Key point of Strings

- Strings are objects in Java.
- When we create a String object, we are creating a string that cannot be changed. In other words, once a String object has been created, we cannot change any characters in string. Therefore, string object is immutable in Java.
- The number of characters in a string is called length of string. For example, the length of “Hello” is 5.
- String class in Java has numerous methods for string manipulation.
- String is not a primitive data type. It is a reference data type.

Strings Buffer

- **StringBuffer** in Java is a peer class of **String** that provides much of the functionality of strings. It provides more flexibility than String.
- As we know that a string is fixed-length, immutable, and cannot be modified. That is, once we create a String object, we cannot change its content.
- To overcome this, Java language introduced another class called StringBuffer. It represents a growable nature and mutable. That means that once we create a StringBuffer class object, we can perform any required changes in the object. i.e, its data can be modified.
- It creates strings of flexible length that can be modified in terms of both length and content.

Strings Buffer



Capacity: The total number of characters hold in the StringBuffer object is called capacity.

Length: The number of characters already present in the StringBuffer object is called length.

Strings vs Sting Buffer

String	StringBuffer
String is immutable.	StringBuffer is mutable.
When we concatenate too many strings, it takes longer and uses more memory because it creates a new instance each time.	When concatenating two strings, StringBuffer is faster and uses less memory.
When executing a concatenation operation, the String class is slower.	When performing concatenation operations, the StringBuffer class is faster.
The equals() method of the Object class is overridden by the String class. As a result, the equals() method can be used to compare the contents of two strings.	The equals() function of the Object class is not overridden by the StringBuffer class.
It overrides the object class's equal() and hashCode() methods.	It cannot overrides the object class's equal() and hashCode() methods.
String constant pool is used by the String class.	Heap memory is used by StringBuffer

Input and Output

- Java input and output consists of elements such as input, output and stream.
- The input is the data that we give to the program.
- The output is the data what we receive from the program in the form of result.
- Stream represents flow of data or the sequence of data.
- To give input we use the **input stream** and to give output we use the **output stream**.

How Input is read from the Keyboard?

- The `"System.in"` represents the keyboard.
- To read data from keyboard it should be connected to `"InputStreamReader"`.
- From the `"InputStreamReader"` it reads data from the keyboard and sends the data to the `"BufferedReader"`. From the `"BufferedReader"` it reads data from `InputStreamReader` and stores data in buffer. It has got methods so that data can be easily accessed.

Reading Input from Console

- Input can be given either from file or keyword. In java, input can be read from console in 3 ways:
 - `BufferedReader`
 - `StringTokenizer`
 - `Scanner`
- **BufferedReader – Java class**
 - `BufferedReader bufferedreader = new BufferedReader(new InputStreamReader(System.in));`
 - `int age = bufferedreader.read();`
 - `String name = bufferedreader.readLine();`

Reading Input from Console

- **String Tokenizer – Java class**
- It can be used to accept multiple inputs from console in a single line where as `BufferedReader` accepts only one input from a line. It uses delimiter (space, comma) to make the input into tokens.
- The syntax is as follows:
 - `BufferedReader bufferedreader = new BufferedReader(new InputStreamReader(System.in));`
 - `String input = bufferedreader.readline();`
 - `StringTokenizer tokenizer = new StringTokenizer(input, ",");`
 - `String name = tokenizer.nextToken();`
 - `int age=tokenizer.nextToken();`

Reading Input from Console

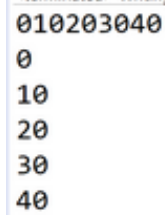
- **Scanner – Java class**
- It accepts multiple inputs from file or keyboard and divides into tokens. It has methods to different types of input (int, float, string, long, double, byte) where tokenizer does not have.
- The syntax is denoted below :
 - `Scanner scanner = new Scanner(System.in);`
 - `int rollno = scanner.nextInt();`
 - `String name = scanner.next();`

Writing console output in java

- In java, there are two methods to write console output.
 - Using `print()` and `println()` methods
 - Using `write()` method
- Writing console output using `print()` and `println()` methods:
 - The `PrintStream` is a built-in class that provides two methods `print()` and `println()` to write console output.
 - Both `print()` and `println()` methods are used with `System.out` stream.
 - The `print()` method writes console output in the same line. This method can be used with console output only.
 - The `println()` method writes console output in a separate line (new line). This method can be used with console and also with other output sources.

Writing console output in java

```
public class WritingDemo {
    public static void main(String[] args) {
        int[] list = new int[5];
        for(int i = 0; i < 5; i++)
            list[i] = i*10;
        for(int i:list)
            System.out.print(i);    //prints in same line
        System.out.println("");
        for(int i:list)
            System.out.println(i); //Prints in separate lines
    }
}
```



010203040

Writing console output in java

- Writing console output using write() method
 - The write() method take integer as argument, and writes its ASCII equivalent character on to the console, it also accept escape sequences.

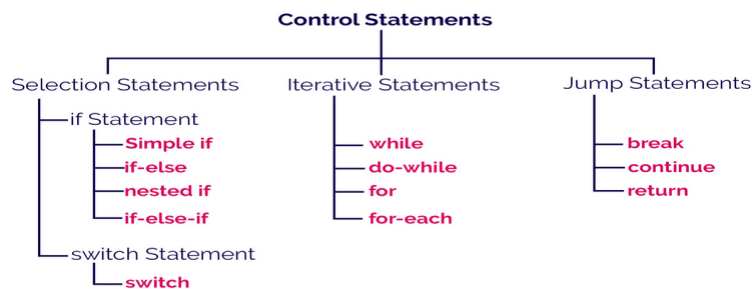
```
public class WritingDemo {
    public static void main(String[] args) {
        int[] list = new int[26];
        for(int i = 0; i < 26; i++) {
            list[i] = i + 65; }
        for(int i:list) {
            System.out.write(i);
            System.out.write('\n'); }
    }
}
```



A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z

Java Control Statements

- In java, the default execution flow of a program is a sequential order.
- But the sequential order of execution flow may not be suitable for all situations. Sometimes, we may want to jump from line to another line, we may want to skip a part of the program, or sometimes we may want to execute a part of the program again and again. To solve this problem, java provides control statements.



Types of Control Statements

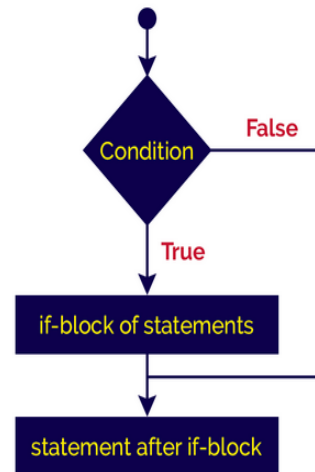
- The control statements are classified as follows.
 - Selection Control Statements (Decision Making Statements)
 - Iterative Control Statements (Looping Statements)
 - Jump Statements
- **Selection Control Statements** : The selection statements are also known as decision making statements or branching statements. The selection statements are used to select a part of the program to be executed based on a condition. Java provides the following selection statements.
 - if statement
 - if-else statement
 - if-elif statement
 - nested if statement
 - switch statement

if statement in java

Syntax

```
if(condition){
    if-block of statements;
    ...
}
statement after if-block;
...
```

Flow of execution



if statement in java Example

```
import java.util.Scanner;

public class IfStatementTest {

    public static void main(String[] args) {
        Scanner read = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int num = read.nextInt();
        if((num % 5) == 0) {
            System.out.println("We are inside the if-block!");
            System.out.println("Given number is divisible by 5!!!");
        }
        System.out.println("We are outside the if-block!!!");
    }
}
```

```
Enter any number: 12
We are outside the if-block!!!

<terminated> IfStatementTest [Java Application] C:\Program File
Enter any number: 20
We are inside the if-block!
Given number is divisible by 5!!
We are outside the if-block!!!
```

if-else statement in java

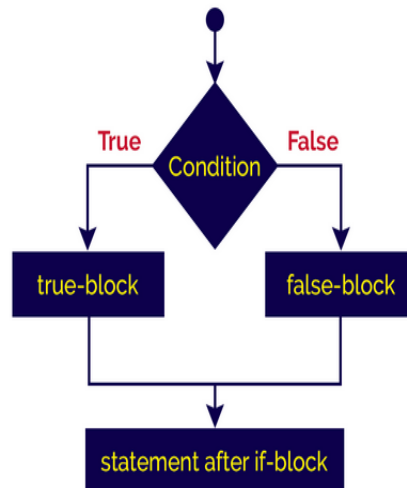
Syntax

```

if(condition){
    true-block of statements;
    ...
}
else{
    false-block of statements;
    ...
}
statement after if-block;
...

```

Flow of execution



if-else statement in java Example

```

import java.util.Scanner;

public class IfElseStatementTest {
    public static void main(String[] args) {
        Scanner read = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int num = read.nextInt();
        if((num % 2) == 0) {
            System.out.println("We are inside the true-block!");
            System.out.println("Given number is EVEN number!!");
        }
        else {
            System.out.println("We are inside the false-block!");
            System.out.println("Given number is ODD number!!");
        }
    }
}

```

if-else statement in java Example

```

    }
        System.out.println("We are outside the if-block!!!");
    }
}

```

```

Enter any number: 16
We are inside the true-block!
Given number is EVEN number!!
We are outside the if-block!!!

```

Nested if-else statement in java

○ Syntax :

```

    if(condition_1){
        if(condition_2){
            inner if-block of statements;
            ...
        }
        ...
    }

```


Nested if-else statement in java example

```
import java.util.Scanner;

public class NestedIfStatementTest {
    public static void main(String[] args) {
        Scanner read = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int num = read.nextInt();
        if (num < 100) {
            System.out.println("\nGiven number is below 100");
            if (num % 2 == 0)
                System.out.println("And it is EVEN");
            else
                System.out.println("And it is ODD");
        } else
```

Nested if-else statement in java example

```
        System.out.println("Given number is not below 100");
        System.out.println("\nWe are outside the if-block!!!");
    }
}
```

```
Enter any number: 75

Given number is below 100
And it is ODD

We are outside the if-block!!!
```

if-else if statement in java

- Syntax :

```

if(condition_1){
    condition_1 true-block;
    ...
}
else if(condition_2){
    condition_2 true-block;
    condition_1 false-block too;
    ...
}

```

if-else if statement in java example

```

import java.util.Scanner;

public class IfElseIfStatementTest {
    public static void main(String[] args) {
        int num1, num2, num3;
        Scanner read = new Scanner(System.in);
        System.out.print("Enter any three numbers: ");
        num1 = read.nextInt();
        num2 = read.nextInt();
        num3 = read.nextInt();
        if( num1>=num2 && num1>=num3)
            System.out.println("\nThe largest number is " + num1);
        else if (num2>=num1 && num2>=num3)

```

if-else if statement in java example

```

        System.out.println("\nThe largest number is " + num2) ;
    else
        System.out.println("\nThe largest number is " + num3) ;
        System.out.println("\nWe are outside the if-block!!!");
    }
}

```

```

<terminated> nestedifstatementtest [Java Application] C:\Program
Enter any three numbers: 17 39 14

The largest number is 39

We are outside the if-block!!!

```

switch statement in java

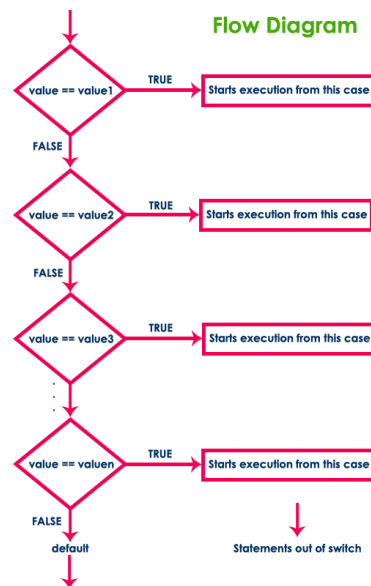
Syntax

```

switch ( expression or value )
{
    case value1: set of statements;
        ....
    case value2: set of statements;
        ....
    case value3: set of statements;
        ....
    case value4: set of statements;
        ....
    case value5: set of statements;
        ....
    .
    .
    default: set of statements;
}

```

Flow Diagram



switch statement in java example

```
import java.util.Scanner;

public class SwitchStatementTest {
    public static void main(String[] args) {
        Scanner read = new Scanner(System.in);
        System.out.print("Press any digit: ");
        int value = read.nextInt();
        switch( value ) {
            case 0: System.out.println("ZERO"); break ;
            case 1: System.out.println("ONE"); break ;
            case 2: System.out.println("TWO"); break ;
            case 3: System.out.println("THREE"); break ;
            case 4: System.out.println("FOUR"); break ;
            case 5: System.out.println("FIVE"); break ;
```

switch statement in java example

```
            case 6: System.out.println("SIX"); break ;
            case 7: System.out.println("SEVEN"); break ;
            case 8: System.out.println("EIGHT"); break ;
            case 9: System.out.println("NINE"); break ;
            default: System.out.println("Not a Digit");
        }
    }
}
```

```
Press any digit: 8
EIGHT
```

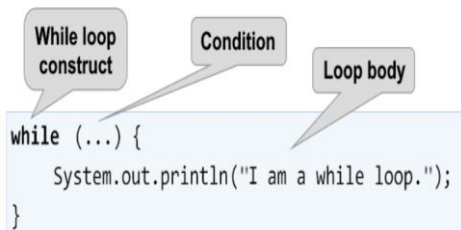
Loops in java

- The java programming language provides a set of iterative statements that are used to execute a statement or a block of statements repeatedly as long as the given condition is true.
- The iterative statements are also known as looping statements or repetitive statements.
- Java provides the following iterative statements.
 - while statement
 - do-while statement
 - for statement
 - for-each statement

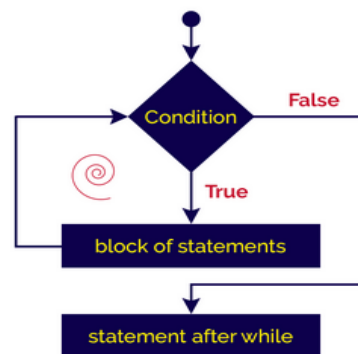
while statement in java

Syntax

```
while(boolean-expression){
    block of statements;
    ...
}
statement after while;
...
```



Flow of execution



- The while statement is also known as **Entry control looping statement**.

while statement in java example

```

1
2
3
4
5
6
7
8
9
10
Statement after while!

public class WhileTest {
    public static void main(String[] args) {
        int num = 1;
        while(num <= 10) {
            System.out.println(num);
            num++;
        }
        System.out.println("Statement after while!");
    }
}

```

do-while statement in java

- The do-while statement is also known as the [Exit control looping statement](#).

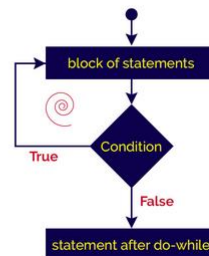
Syntax

```

do{
    block of statements;
}while(boolean-expression);
statement after do-while;
...

```

Flow of execution



do-while statement in java example

```

public class DoWhileTest {
    public static void main(String[] args) {
        int num = 1;
        do {
            System.out.println(num);
            num++;
        }while(num <= 10);
        System.out.println("Statement after do-while!");
    }
}

```

1
2
3
4
5
6
7
8
9
10
Statement after do-while!

for statement in java

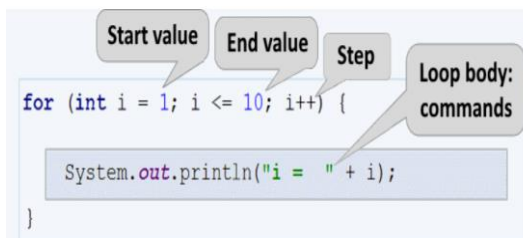
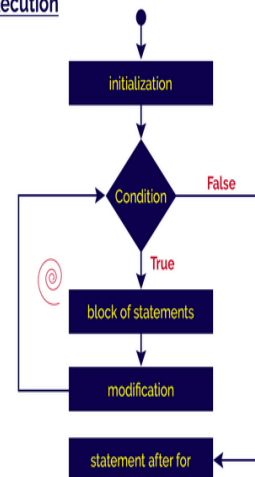
Syntax

```

for(initialization; boolean-expression; modification){
    block of statements;
}
...
statement after for;
...

```

Flow of execution



for statement in java example

```
public class ForTest {
    public static void main(String[] args) {
        for(int i = 0; i < 10; i++) {
            System.out.println("i = " + i);
        }
        System.out.println("Statement after for!");
    }
}
```

```
i = 0
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
Statement after for!
```

for-each statement in java

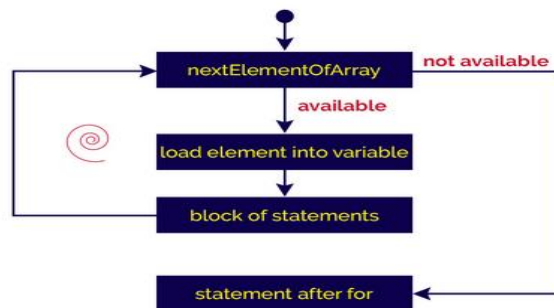
- The Java for-each statement was introduced since Java 5.0 version.
- It provides an approach to traverse through an array or collection in Java.
- The for-each statement also known as enhanced for statement.
- The for-each statement executes the block of statements for each element of the given array or collection.
- **Note:** In for-each statement, we can not skip any element of given array or collection.

for-each statement in java

Syntax

```
for( dataType variableName : Array ){
    block of statements;
    ...
}
statement after for;
...
```

Flow of execution



for-each statement in java example

```
public class ForEachTest {
    public static void main(String[] args) {
        int[] arrayList = {10, 20, 30, 40, 50};
        for(int i : arrayList) {
            System.out.println("i = " + i);
        }
        System.out.println("Statement after for-each!");
    }
}
```

```
i = 10
i = 20
i = 30
i = 40
i = 50
Statement after for-each!
```



115